

STM32 HAL GPIO Button-Controlled LED Blink

Project Overview

This project demonstrates how to control an LED using a push button with the **STM32 HAL (Hardware Abstraction Layer) library**.

When the push button is pressed, the STM32 detects the button state and calls a custom function that makes the LED blink **10 times**.

When the button is released, the LED remains **OFF**.

This project is a basic example of **GPIO input and GPIO output control using STM32 HAL**.

Hardware Required

- STM32F407 microcontroller/board
- LED
- Resistor for LED
- Push button
- Jumper wires
- USB/ST-LINK programmer

Pin Configuration

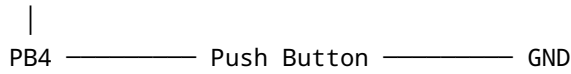
Component	STM32 Pin	Configuration
LED	PA15	GPIO Output Push-Pull
Push Button	PB4	GPIO Input with Pull-Up

LED

STM32 PA15 — Resistor — LED — GND

Push Button

3.3V
|
Internal Pull-Up



Because the internal pull-up resistor is enabled:

Button Released → PB4 = HIGH (1)
Button Pressed → PB4 = LOW (0)

Therefore, the button is considered **pressed when the input reads** `GPIO_PIN_RESET`.

STM32CubeMX / CubeIDE Configuration

1. Configure LED Pin

Configure:

Pin: PA15
Mode: GPIO_Output
Output Type: Push-Pull
Pull: No Pull
Speed: Low
Initial Output Level: LOW

2. Configure Button Pin

Configure:

Pin: PB4
Mode: GPIO_Input
Pull: Pull-Up

The internal pull-up keeps PB4 HIGH when the button is not pressed.

When the button is pressed, PB4 is connected to GND and becomes LOW.



GPIO Definitions

The following macros are used to make the code easier to understand:

```
#define btn      GPIOB
#define btn_pin  GPIO_PIN_4

#define led      GPIOA
#define led_pin  GPIO_PIN_15
```

Instead of writing:

```
HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4);
```

we can write:

```
HAL_GPIO_ReadPin(btn, btn_pin);
```

Similarly:

```
HAL_GPIO_WritePin(led, led_pin, GPIO_PIN_SET);
```

is equivalent to:

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_SET);
```



Button Detection

The button state is checked using:

```
HAL_GPIO_ReadPin(btn, btn_pin)
```

The condition:

```
if (HAL_GPIO_ReadPin(btn, btn_pin) == GPIO_PIN_RESET)
```

means:

```
PB4 = LOW
    ↓
Button is pressed
    ↓
Execute LED blinking function
```

The `else` condition means that the button is released:

```
else
{
    HAL_GPIO_WritePin(led, led_pin, GPIO_PIN_RESET);
}
```

Therefore:

```
Button Pressed → LED blinks 10 times
Button Released → LED OFF
```



LED Blink Function

A separate user-defined function called `led_fun()` is used for LED blinking.

```
void led_fun()
{
    for (int i = 0; i < 10; i++)
    {
        // LED ON
        HAL_GPIO_WritePin(led, led_pin, GPIO_PIN_SET);
        HAL_Delay(200);

        // LED OFF
        HAL_GPIO_WritePin(led, led_pin, GPIO_PIN_RESET);
        HAL_Delay(200);
    }
}
```

The `for` loop executes 10 times.

Each cycle consists of:

LED ON → 200 ms
LED OFF → 200 ms

Therefore:

ON OFF ON OFF ON OFF ... 10 times

The total time is approximately:

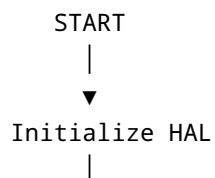
$10 \times (200 \text{ ms} + 200 \text{ ms})$
 $= 10 \times 400 \text{ ms}$
 $= 4000 \text{ ms}$
 $= 4 \text{ seconds}$

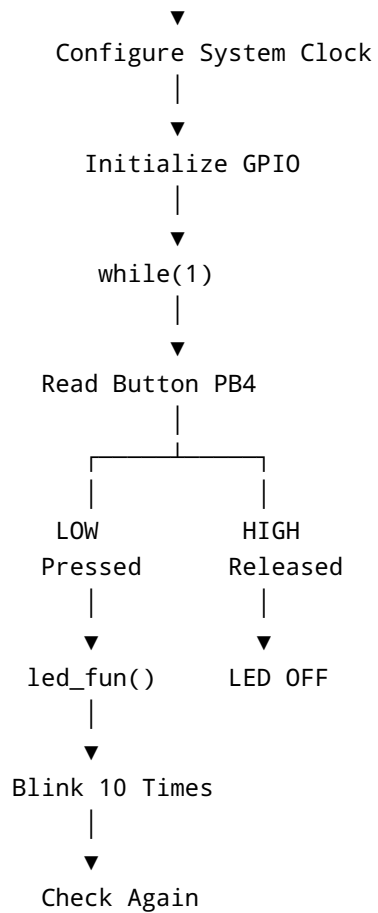
Main Program Logic

The main loop continuously checks the button:

```
while (1)
{
    if (HAL_GPIO_ReadPin(btn, btn_pin) == GPIO_PIN_RESET)
    {
        led_fun();
    }
    else
    {
        HAL_GPIO_WritePin(led, led_pin, GPIO_PIN_RESET);
    }
}
```

The program flow is:





Level Trigger Concept

This project uses **level-based button detection**.

The STM32 continuously reads the current state of PB4:

```
HAL_GPIO_ReadPin(btn, btn_pin)
```

The logic is:

```
PB4 = LOW
↓
Button currently pressed
↓
Run LED function
```

This is different from an **edge-triggered interrupt**.

Level Trigger

"Is the button currently pressed?"

Edge Trigger

"Did the button state just change?"

The current project uses **level triggering through polling**.

Important Note About Current Code

The function:

```
led_fun();
```

uses:

```
HAL_Delay(200);
```

This means the CPU is blocked inside the function while the LED is blinking.

For example:

```
Button Pressed
  ↓
led_fun()
  ↓
Blink 1
  ↓
Delay
  ↓
Blink 2
  ↓
Delay
  ↓
...
```

```
↓  
Blink 10  
↓  
Return to while(1)
```

During this time, the main loop is not continuously checking the button.

This is acceptable for a **basic GPIO learning project**, but for advanced embedded applications such as drone flight controllers, motor control, UART communication, and sensor processing, a **non-blocking approach** using `HAL_GetTick()` or **hardware timers** is preferred.

HAL Functions Learned

This project demonstrates the following STM32 HAL functions:

1. Initialize HAL

```
HAL_Init();
```

Initializes the HAL library and basic MCU peripherals.

2. Read GPIO Input

```
HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_4);
```

Reads the current logic level of PB4.

3. Write GPIO Output

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_SET);
```

Sets PA15 HIGH.

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_15, GPIO_PIN_RESET);
```

Sets PA15 LOW.

4. Delay

```
HAL_Delay(200);
```

Creates a delay of 200 milliseconds.

5. GPIO Initialization

```
HAL_GPIO_Init();
```

Initializes GPIO pins according to the configuration.



Future Improvements

This project can be extended with:

- ☐ Button debounce
 - ☐ Edge-triggered EXTI interrupt
 - ☐ Rising-edge detection
 - ☐ Falling-edge detection
 - ☐ Multiple LED patterns
 - ☐ SOS Morse code
 - ☐ Heartbeat LED pattern
 - ☐ Pattern selection using a button
 - ☐ Non-blocking LED blinking using `HAL_GetTick()`
 - ☐ Timer-based LED control
 - ☐ Multiple button inputs
 - ☐ PWM-based LED brightness control
-



Learning Outcome

After completing this project, you should understand:

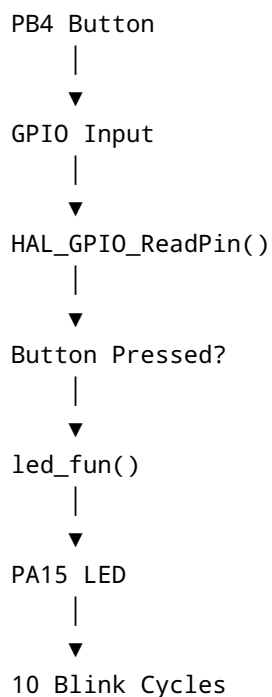
- GPIO output configuration
- GPIO input configuration
- Internal pull-up resistor
- Button logic
- HIGH and LOW logic
- `GPIO_PIN_SET`

- `GPIO_PIN_RESET`
- `HAL_GPIO_ReadPin()`
- `HAL_GPIO_WritePin()`
- User-defined functions
- `for` loops
- Level-triggered GPIO polling
- Basic LED pattern generation

Summary

This project is a basic introduction to **STM32 GPIO programming using the HAL library**.

The push button connected to **PB4** is configured with an internal pull-up resistor. When the button is pressed, PB4 becomes LOW. The STM32 detects this LOW level and calls `led_fun()`, causing the LED connected to **PA15** to blink 10 times.



Project Type: STM32 HAL GPIO

Microcontroller: STM32F407

Programming Language: Embedded C

Development Environment: STM32CubeIDE

HAL Library: STM32F4 HAL